

Urban Mobility Data Explorer

Technical Report | NYC Taxi Trip Dashboard

Aurore Irakarama, Huguette Uwase, Lorris Hira, Nziza Samuel, Bruce Eddy Manzi

1. Problem Framing and Dataset Analysis

Our project uses NYC Taxi & Limousine Commission yellow taxi trip records for January 2019. We combined them with zone and map files to connect trips to real locations such as boroughs and neighborhoods. The dashboard helps users understand travel patterns, trip costs, and differences between boroughs.

Data Cleaning

Two zones had missing information, so we labeled them as "Unknown" because trips still used them. Missing congestion surcharge values were treated as no surcharge.

Invalid trips were removed:

Reason for exclusion	Rows removed
Passenger count invalid	1,286
Zero or negative distance	1,028
Pickup date not in January 2019	324
Zero or negative fare	186
Invalid pickup/dropoff times	88
Trip longer than 24 hours	1

We only kept trips from January 2019 and limited passenger counts to valid taxi capacity.

An unexpected finding was that invalid passenger counts were the most common issue, showing that passenger data is less reliable than expected.

2. System Design Decisions

The data is cleaned using a Python pipeline, stored in PostgreSQL, accessed through a Flask API, and displayed in a JavaScript dashboard using Chart.js.

Technology Choices

- Flask was chosen because our data processing was already done in Python.
- PostgreSQL was selected to maintain relationships and data consistency.
- Plain JavaScript and Chart.js were sufficient for a dashboard with fixed charts and filters.

Database Design

The schema includes:

- Reference tables (vendors, rate codes, payment types, zones)
- Trips table
- Trip_features table containing calculated values
- Excluded_records_log table for failed records

Trade-offs

A normalized schema requires JOIN operations but improves data integrity. Data is loaded in batches of 5,000 rows, with fallback to individual inserts when needed.

3. Algorithmic Logic and Data Structures

To identify the busiest pickup zones, we built a custom ranking method.

Trips are counted by pickup zone using a dictionary. Zones are then ranked from highest to lowest pickup count through repeated comparisons and swaps.

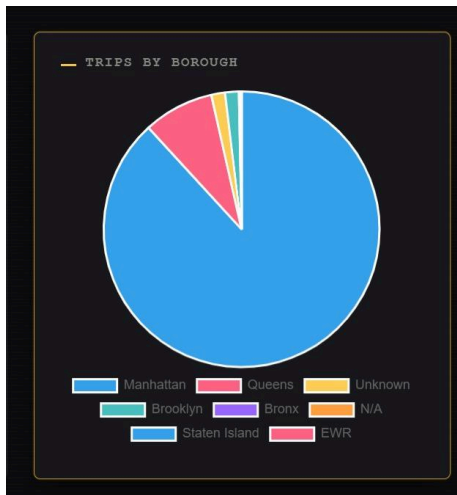
Complexity

Counting takes time proportional to the number of trips. Ranking takes time proportional to the square of the number of zones. Space usage depends only on the number of zones.

4. Insights and Interpretation

The dashboard focuses on:

Pickup Zones



High-demand zones indicate areas where many people use taxis, often around business districts and transport hubs.

Borough Comparison

```
USER@DESKTOP-COICPB5 MINGW64 ~/Downloads/damn/summative/urban-mobility-data-explorer/backend (main)
$ cp ../cleaned_tripdata_subset.csv cleaned_enriched_data.csv
$ python3 ranking_algorithm.py
Rank 1: Zone 186 - 19638 trips
Rank 2: Zone 237 - 18831 trips
Rank 3: Zone 132 - 18556 trips
Rank 4: Zone 161 - 18549 trips
Rank 5: Zone 236 - 18094 trips
Rank 6: Zone 48 - 17866 trips
Rank 7: Zone 230 - 16438 trips
Rank 8: Zone 138 - 15423 trips
Rank 9: Zone 162 - 15414 trips
Rank 10: Zone 170 - 15307 trips
Rank 11: Zone 142 - 14434 trips
Rank 12: Zone 239 - 13472 trips
Rank 13: Zone 234 - 12603 trips
Rank 14: Zone 141 - 12444 trips
Rank 15: Zone 163 - 12180 trips
Rank 16: Zone 79 - 12080 trips
Rank 17: Zone 164 - 11913 trips
Rank 18: Zone 68 - 11296 trips
Rank 19: Zone 100 - 11096 trips
Rank 20: Zone 107 - 11006 trips
```

Manhattan is expected to have the most taxi activity. Differences may indicate unequal taxi availability.

Fare by Time of Day



Higher average fares during evenings or nights may reflect longer trips or heavier traffic.

5. Reflection and Future Work

Technical Challenges

We experienced setup issues including missing Flask installations, port conflicts, and frontend errors. We also need to verify that existing zones appear correctly in rankings.

Team Challenges

The project required a strict sequence: data cleaning, schema design, API development, and frontend integration.

Future Work

- Use the `zone_geometry` table to build an interactive map.
- Add automated tests for cleaning rules and ranking logic.
- Add monitoring for new data loads.
- Add query caching to support more users.